

Early Estimation in Agile Software Development Projects: A Systematic Mapping Study

José Gamaliel Rivera Ibarra ^{1,†} , Gilberto Borrego ^{1,*,†}  and Ramón R. Palacio ^{2,†} 

¹ Departamento de Computación y Diseño, Instituto Tecnológico de Sonora, Ciudad Obregón 85000, Mexico; joseg.rivera@potros.itson.edu.mx

² Unidad Navojoa, Instituto Tecnológico de Sonora, Navojoa 85860, Mexico; ramon.palacio@itson.edu.mx

* Correspondence: gilberto.borrego@itson.edu.mx; Tel.: +52-644-410-9000

† These authors contributed equally to this work.

Abstract: Estimating during the early stages is crucial for determining the feasibility and conducting the budgeting and planning of agile software development (ASD) projects. However, due to the characteristics of ASD and limited initial information, these estimates are often complicated and inaccurate. This study aims to systematically map the literature to identify the most used estimation techniques; the reasons for their selection; the input artifacts, predictors, and metrics associated with these techniques; as well as research gaps in early-stage estimations in ASD. This study was based on the guidelines proposed by Kitchenham for systematic literature reviews in software engineering; a review protocol was defined with research questions and criteria for the selection of empirical studies. Results show that data-driven techniques are preferred to reduce biases and inconsistencies of expert-driven techniques. Most selected studies do not mention input artifacts, and software size is the most commonly used predictor. Machine learning-based techniques use publicly available data but often contain records of old projects from before the agile movement. The study highlights the need for tools supporting estimation activities and identifies key areas for future research, such as evaluating hybrid approaches and creating datasets of recent projects with sufficient contextual information and standardized metrics.

Keywords: systematic mapping; literature review; agile development; project estimation; early stage



Citation: Rivera Ibarra, J.G.; Borrego, G.; Palacio, R.R. Early Estimation in Agile Software Development Projects: A Systematic Mapping Study. *Informatics* **2024**, *11*, 81. <https://doi.org/10.3390/informatics11040081>

Academic Editor: Francisco José García-Peñalvo

Received: 2 October 2024

Revised: 29 October 2024

Accepted: 30 October 2024

Published: 4 November 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Software development is a significant economic activity that results in annual expenditures exceeding one billion dollars for companies worldwide [1]. These investments are driven by the need to increase productivity, automate tasks, and enable business transformation to maintain competitiveness. Companies can acquire software through packaged products (Off-the-Shelf), custom software development services, or a combination.

Particularly, custom software development involves designing and constructing software that fulfill a company's needs, usually in the form of web or mobile applications. These development efforts are organized as projects comprising a series of tasks carried out by a team within a set timeframe, all aimed at delivering the software and related artifacts with a certain level of quality. The success of such projects is typically measured against key objectives, including scope, duration, and cost [2]. However, a significant percentage of projects fail to meet these goals, often due to inadequate planning and inaccurate estimation [3–10].

Effective project management is essential to navigating these challenges. It involves systematically planning, organizing, directing, and controlling the resources and activities necessary to achieve project goals [11]. The Project Management Institute (PMI) offers guidance through its Project Management Body of Knowledge (PMBOK). It provides a structured framework for managing projects and categorizes knowledge into ten key areas

and five process groups. Among these, the planning process group is particularly crucial as it sets clear objectives, identifies risks, and allocates resources efficiently [12]. Successful planning is built on experience, knowledge, and precise estimation, which enable informed decisions about timelines, costs, and customer expectations. While traditional project estimates are rarely exact due to evolving objectives and scope during execution, effective project management ensures that adjustments remain within acceptable limits, leading to outcomes that closely align with initial projections [13].

Project estimation, a critical planning component, involves predicting the effort, duration, and cost required to develop software. This task demands a deep understanding of the software development process and the characteristics, constraints, specifications, and other key aspects of the software product being built. An early estimation is a preliminary calculation performed before the project begins; it is usually based on high-level requirements and is used mainly for contract negotiation and general planning [3,11,14]. Inaccurate initial estimation can have negative consequences, such as losing contracts due to overestimation and facing missed deadlines, low product quality, customer dissatisfaction, penalties, and project cancellation due to underestimation [3,15,16].

Over the years, estimating development projects has been a major challenge in software engineering. Many techniques have been proposed to improve the accuracy of estimation. Factors such as technological advances and changes in development methodologies affect the evolution of estimation techniques. One of the most significant changes in software development is adopting the agile approach, which consists of a set of techniques, methods, and practices guided by the values and principles of the Agile Manifesto [17].

Agile software development (ASD) is iterative and incremental, allowing quick adaptation to changes and prioritizing customer satisfaction by continuously delivering working software. The agile movement has introduced new estimation techniques, such as Planning Poker, T-shirt Size, Dot Voting, and Bucket System [18], which are commonly used for creating work plans at the release, iteration, or workday level during the project execution phase [19]. Although the Agile approach allows requirements to evolve throughout the project, an initial estimate helps the client determine whether the available budget is sufficient to cover critical software requirements, which guides the prioritization and planning of future releases. In addition, Agile projects often have deadlines; an accurate initial estimate allows the development company to understand the associated effort, set realistic resource expectations, and create a viable work plan to meet these objectives within the agreed-upon timeline.

Initial estimation, conducted during the early stages of a project (in the initiation and planning phases, according to the PMBOK [11]), is particularly critical as it forms the basis for developing customer contracts and making key decisions during project execution [20,21]. However, initial estimates are often inaccurate [22], primarily due to a lack of detailed information, ambiguous requirements, and human and technical factors such as excessive optimism, limited knowledge, and insufficient historical data [23–26]. These challenges are further compounded by the fact that, guided by the Agile Manifesto [17], agile development teams prioritize construction activities over those not perceived as valuable to the customer, such as obtaining metrics and exhaustive documentation, including detailed work plans. This is because agile methods are often employed in innovative projects where requirements are uncertain and typically discovered during development. While creating detailed work plans may seem counter to the agile philosophy—since agile methods promote adaptability to changing requirements and priorities—a detailed plan provides stakeholders with a comprehensive view of the project's scope, resources, and necessary activities.

In addition, detailed planning facilitates the identification of priorities and more efficient coordination of work [27]. At the same time, the collection of metrics provides objective, quantitative data on resources, time, and effort expended. Together, detailed planning and metrics allow teams to understand the discrepancies between estimated and actual effort in project tasks, which provides a solid foundation for evaluating new projects,

developing predictive models, and identifying patterns and trends, ultimately improving the accuracy of future project estimates [19,28].

Given these complexities, early estimation of agile software projects remains critical and challenging. This study examines the main facets of estimation techniques found in the literature that tackle these challenges. A systematic literature review is necessary to collect and categorize existing information to gain a comprehensive understanding of current knowledge on early-stage estimation in ASD projects. This helps to identify patterns, areas requiring further investigation, and potential subjects for future research.

This study aims to present a comprehensive and organized overview of the most frequently utilized estimation techniques for the early stages of agile projects, offering a solid reference for future research. This paper is organized as follows: Section 2 describes some concepts used in this study. Section 3 presents related work, Section 4 describes the details of the methodology used, Section 5 introduces the research results, and Section 6 discusses the findings. Finally, Section 7 identifies the threats to validity, Section 8 presents conclusions about this work, and Section 9 suggests for future work.

2. Background

In this section, we address some fundamental aspects of the estimation process to provide an overview for understanding and analyzing the literature about early estimations in agile projects.

2.1. Estimation Process

The primary purpose of the estimation process in software development is to predict the investment required to complete a project. This investment is derived from two key factors [11]:

- The human and technological resources needed, including team salaries, software licenses, service fees, and external suppliers.
- The duration for which these resources will be required.

The estimation process takes input from various artifacts, such as documents, diagrams, or prototypes, which outline the software's characteristics, standards, constraints, and other requirements. Participants analyze these artifacts to identify the tasks necessary to build the software based on their development process. Since tasks vary in complexity and size, the amount of work, or effort, needed to complete each task also differs [11].

The time required to finish a task depends on the capacity of the team or individual assigned to it. Capacity refers to how much work a person or team can accomplish within a given period. Therefore, the time to complete a task is influenced by the effort it demands and the capacity of the team carrying it out [11].

Throughout the estimation process, the goal is to forecast the total effort, which is the sum of all task efforts in the project. In software development, these tasks typically involve activities like documentation, design, implementation, testing, and deployment [19]. Consequently, total effort is closely linked to the size and complexity of the software being developed.

2.2. Estimation Techniques Approaches

Development companies generally rely on one or more techniques to guide their estimation process. Over time, different approaches to estimation have emerged, each offering unique benefits. The main categories of estimation techniques are as follows:

- Expert-driven techniques: These rely on the experience of the participants in the estimation process to predict the total effort of the project. Agile methodologies often favor expert-driven techniques due to their simplicity, flexibility, and adaptability to project-specific conditions. However, their accuracy is highly dependent on the experience of the estimators and can be prone to bias and subjectivity. Without a

systematic foundation, these methods may yield variable accuracy. Popular techniques in this category include Planning Poker and Wideband Delphi [29].

- Data-driven techniques: These use quantitative data and metrics from past projects to predict project effort. These techniques offer a more objective and data-grounded approach by basing estimates on historical results. Their main drawback is the need for accurate historical data, which may not always be available, and the complexity of constructing predictive models. Data-driven techniques can be sub-categorized into algorithmic techniques such as COCOMO II and Function Points, and machine learning-based techniques [30].

Data-driven techniques often employ mathematical models, statistical methods, or machine learning algorithms to predict effort, duration, and cost—known as dependent variables. These models are built using historical data, which serve as effort predictors. To construct these models, a dataset is gathered, relevant predictors are selected, and those with significant correlations to the dependent variables are used. Common predictors include software size, team experience, and project domain. The dataset is split into training and validation subsets: the former for building the models or training algorithms and the latter for testing and validating their accuracy [28].

2.3. Software Size Metrics

Software size is one of the most frequently used predictors in data-driven techniques. It quantifies the magnitude of a software product, often based on its functionality or lines of code. Common methods for measuring software size include the following [27]:

- Lines of Code (LoC): this involves counting the number of lines in the software's source code.
- Function Points (FP): measures the size of software based on the functionality it provides from the user's perspective, typically in terms of data inputs and outputs.
- Use Case Points (UCP): similar to Function Points, but focused on Use Cases as the unit of measurement.
- Story Points (SP): while Story Points do not directly measure software size, they reflect the relative effort required to implement a User Story and can serve as a proxy for software size.

3. Related Work

In this section, we compare and analyze relevant literature about review studies on estimation in the early stages of ASD projects. We highlight the knowledge gaps that have been identified and emphasize the need and importance of realizing this study.

Although multiple studies exist in the literature on state-of-the-art estimation in agile development projects [19,31–37], these do not address the issues and features related to estimating during the early stages of the agile development project life cycle. In fact, Usman et al. [19] established that, in the context of ASD, estimates are used to plan at three primary levels: release, iteration, or daily work, without considering the estimation made at the beginning of the project prior to its execution for budgeting and planning purposes.

Only a few literature review studies address estimations in projects' early stages regarding the ASD life cycle. However, these works focus on specific aspects without providing a general perspective of the topic, as is the purpose of the present study. Below is a brief analysis of these studies.

In 2020, Yasir Mahmood et al. [8] conducted a systematic review of the literature on the use of Use Case Points (UCP) and estimation techniques based on expert judgment to predict effort in software development. They focus on understanding the datasets' characteristics and the commonly used metrics for estimation and determining whether using UCP and expert judgment allows favorable results to be achieved in estimating ASD projects. They report that the studies they reviewed use estimation techniques with varied approaches, which they classify as algorithmic, non-algorithmic, and based on machine learning. Still, these studies, to a lesser extent, use UCP and expert judgment.

Yasir Mahmood et al. consider whether combining UCP with techniques based on expert judgment can help improve the precision of estimates and highlight the need and importance of the correct selection, standardization, and validation of data used to create and evaluate estimation models.

Another study was carried out in 2021 by Mohammad Azzeh et al. [38]. They conducted a literature review on the use of UCP as an estimation method with the aim of classifying the studies according to their contribution, type of dataset, and estimation techniques used in conjunction with UCP. They highlight that most selected studies aim to improve and validate UCP-based models, focusing on the evaluation and comparison against other estimation techniques. They mentioned that UCP has been used with different techniques, mainly fuzzy logic, neural networks, and linear regression. They also discuss and debate the quality and validity of the data used by the authors to build and validate the models that support techniques based on machine learning. Finally, they consider it essential to develop models and automated tools that ease the conversion of use case diagrams to an estimate in UCP as a metric.

Finally, in 2022, Bashaer Alsaadi and Kawther Saeedi [12] conducted a literature review on estimation using data-driven techniques combined with User Stories. Their objective was to identify the estimation methods, their performance, the predictors, and the characteristics of the datasets used for data-driven estimation techniques. They obtained that the use of data-driven techniques has been positively evaluated, showing its effectiveness with User Stories; however, they indicate that more research is required to determine the most appropriate evaluation methods and build specific datasets for User Stories.

Table 1 shows the facets of early-stage estimation addressed by the studies described above. The focus of the literary research has been on empirical studies using the metrics of UCPs [8,38] and User Stories [12], but other metrics such as Function Points were not included. UCPs are studied in combination with expert judgment technique in Mahmood et al. [8], while in Azzeh et al. [38] it is used as an estimation method. User Stories are studied in combination with data-driven techniques in Alsaadi et al. [12]. Only in Alsaadi et al. [12] do they study and classify the predictors. The three previous studies analyzed the characteristics of the datasets and agree on the need to generate updated and standardized databases. None of these studies mention the characteristics of the input artifacts used in the estimation process.

Table 1. Facets of early-stage estimation of agile projects addressed by literature review studies.

Facets	Mahmood et al. [8]	Azzeh et al. [38]	Alsaadi et al. [12]
Input artifacts	No	No	No
Estimation approaches	Expert-driven	Data-driven	Data-driven
Metrics	Use Case Points	Use Case Points	User Stories
Predictors	No	No	Yes
Datasets	Yes	Yes	Yes

Related studies provide valuable insights into estimation techniques in ASD projects, mainly using UCPs and User Stories as software size metrics. However, there is still a need to broaden the scope of these studies to include other software size metrics, such as Function Points, encompassing estimation techniques with different approaches and other vital aspects, such as predictors and input artifacts. A comprehensive review covering estimation techniques of any approach, different size metrics, and emphasizing input artifacts and predictors used during estimates in the early phases of the life cycle of ASD projects is needed.

Our study aims to fill this gap by systematically mapping the literature on early-stage estimation in ASD projects, considering empirical studies on a wide range of estimation approaches, input artifacts, predictors, and the datasets' characteristics. Unlike previous reviews, this study provides a general perspective by examining well-established techniques and emerging trends, focusing on early-stage activities that are critical for project

budgeting and planning. We provide a clearer understanding of the estimation and offer a unique contribution that underscores the importance of early-stage estimations and their impact on the overall success of agile software development projects. The following section presents details on the methodology used to map the literature on this important topic.

4. Research Method

This study aims to analyze, synthesize, and organize the relevant evidence in the literature on early-stage estimation of ASD projects. Our objective is to identify patterns, relationships, and gaps between the topics covered in the literature and establish a starting point for future research in areas that still need to be explored.

From a general review of the literature [19,31–37] on estimates in ASD projects, we find the following:

- Studies on estimation in agile development projects are aimed at evaluating different facets of the estimation process, among which the following stand out:
 - The study of estimation techniques or models.
 - The identification and evaluation of independent variables used by the estimation models.
 - The analysis of the characteristics of the datasets used to create and validate these models.
 - The use of estimation techniques in different application contexts; for example, the impact on estimating global projects with remote teams.
 - The evaluation of the precision and comparison of the performance of the different estimation techniques.
- Studies on this topic mainly focus on aspects of the estimation process related to planning at release, iteration, or daily work level in the context of ASD projects.

This study aims to cover the various facets related to estimation in the early stages of ASD projects, rather than focusing on any single aspect in depth. Since empirical studies on early-stage estimation encompass a range of techniques, metrics, datasets, and contexts, a Systematic Mapping Study (SMS) is better suited to organize and synthesize the extensive and diverse literature on this topic more effectively than a traditional Systematic Literature Review (SLR).

The present study is primarily based on the guidelines for conducting systematic literature reviews in software engineering proposed by Kitchenham et al. [39] and the update by Petersen et al. [40]. According to these guidelines, the SMS process consists of three main phases, each composed of activities. Figure 1 shows a graphic representation of the systematic mapping process adapted from the earlier proposals. Three large blocks (shaded in gray) can be seen in them, representing the phases and the sequence of activities that compose them within each of them. The following subsections describe the most relevant characteristics of these activities, except for the first activity of the Planning phase, “Objective”, which was described at the beginning of this section.

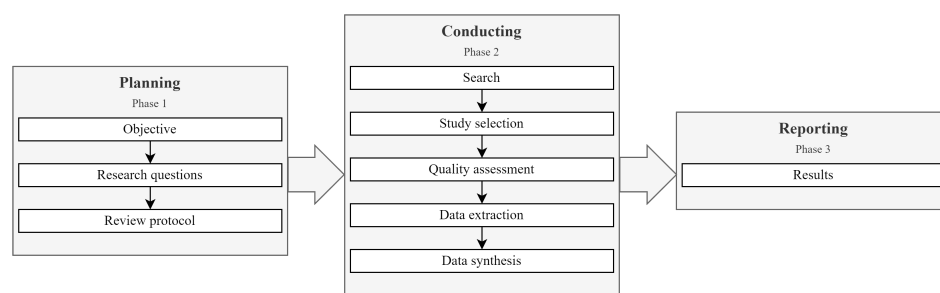


Figure 1. Phases and main activities of the systematic mapping process used in this study.

4.1. Research Questions

The research questions were defined to align with the objectives of this study and address critical issues related to the topic under investigation. The research questions are listed below.

- RQ1: What input artifacts are used to estimate the early stages of agile software development projects?
- RQ2: What estimation approaches and techniques are used to perform estimation in the early stages of agile software development projects?
- RQ3: What predictors are used to estimate in the early stages of agile development projects?
- RQ4: What are the characteristics of the datasets used by estimation techniques in the early stages of agile development projects?

4.2. Review Protocol

The review protocol includes a search strategy with the following main components: (1) definition of the search terms, (2) structuring of the search string, and (3) selection of digital libraries. Additionally, criteria were established to objectively select and evaluate the relevant studies related to the research topic. The details of this review protocol are described below.

4.2.1. Search Terms

Following the guidelines proposed by Petersen [40], we applied the PICOC strategy to determine the keywords of the search string. Since this study does not aim to evaluate the results obtained by the studies or perform a specific comparison between estimation techniques, the Comparison and Outcome criteria of PICOC were omitted (as was suggested by Kitchenham [39]). Based on the above, the PICOC criteria for this study are the following:

- Population: Empirical studies of agile software development projects.
- Intervention: Estimation techniques for agile software projects.
- Context: Estimates made before project execution to support contract negotiation and initial planning.

As suggested by Kitchenham [39], the search terms were refined iteratively, i.e., the initial keywords identified through the PICOC strategy were used in the search engines of digital libraries to understand the volume of results and to identify synonyms and other related terms commonly used by authors. Table 2 shows the keywords and their synonyms or associated terms identified from the results of the initial test searches.

Table 2. Keywords list, synonyms and related terms.

Keywords	Synonyms and Related Terms
software	software development
agile	agile software development, agile methods, agile practices, scrum, extreme programming, xp, kanban, lean, lsd
estimation	prediction, measurement, forecast
technique	method, approach, model
effort	cost, size
predictors	cost-drivers, factors, parameters
accuracy	precision

4.2.2. Search String

The search string was constructed from the keywords listed in Table 2. It is worth mentioning that some keywords were not included in the final search string for the reasons described below. Considering that the search string must match the title, abstract, or keywords of the studies, after conducting some test searches, we found the following issues:

- Since the present study does not focus on a particular agile method, using “agile” allowed us to obtain more results than synonyms and related terms associated with this keyword. According to the search engine guidelines, the results were more restricted even when we used the “OR” operator.
- When we used the keywords “estimate” and “effort” separately in test searches, we found that terms such as “cost estimate”, “effort estimate”, or “size estimate” are frequently used. Therefore, we used these terms instead of the keywords separately to avoid some results unrelated to this research’s interests.
- We found that using the word “predictor” or one of its synonyms (through the “OR” operator) decreased the volume of results considerably. This was because this word was infrequently mentioned in the title, abstract, or keywords, because predictors were not the main focus of many studies. A similar situation occurred with the words “technique” and “precision”. Therefore, we decided to omit these keywords from the search string to increase the number of results.
- Finally, other terms, such as “duration”, were not included because they were not commonly used. We assume that this is because project duration involves estimating the effort first and that its value (duration) depends on the team’s capacity or velocity because it must be calculated for each project.

Additionally, the test searches revealed that the highest percentage of results (over 75%) were published after 2012. This trend can be attributed to the fact that, although agile methods originated in the 1990s, it was not until several years after the Agile Manifesto [17] that they gained wider recognition and adoption by practitioners. This shift led to significant changes in the dynamics of the software development process, creating a need for new estimation techniques that align with the new practices.

It is also noteworthy that, during the test searches, when we used keywords related to the early stages of the projects’ life cycle, the search results decreased considerably. Anticipating that the studies might not include these words in the title, abstract, or keywords, we decided to leave them out of the search string to avoid omitting studies that could be relevant to this research. Finally, the search string was structured as shown below.

“software” AND “agile” AND (“cost estimate” OR “effort estimate” OR “size estimate”).

The keywords “software” and “agile” indicate a focus on agile software development, while the terms “cost estimate”, “effort estimate”, or “size estimate” narrow the results to issues related to project estimation.

4.2.3. Digital Libraries

We reviewed the related works (secondary studies [8,12,38]) on estimation in ASD projects to select the digital libraries to conduct our SMS. The selected digital libraries for use in this study are listed below:

- ACM Digital Library;
- IEEE Xplore;
- ScienceDirect (Elsevier);
- Springer;
- Wiley.

It is essential to mention that the search engines of each digital library support different parameters, limitations, and formats for search strings. Therefore, the search string was adapted to the particular features of each search engine without altering its meaning.

4.3. Studies Selection

The study selection process was conducted in two phases: (1) screening by title and abstract review; (2) screening by full-text review. The inclusion and exclusion criteria listed below were applied in both phases.

4.3.1. Inclusion Criteria

The articles obtained from the searches must meet the criteria listed below to be candidates for this study.

- Must be written in the English language;
- Must have been published after January 2012 (inclusive);
- Must be a primary study on “early-stage estimation of agile software development projects”;
- Must have been published in a journal, conference, or workshop.

4.3.2. Exclusion Criteria

Articles obtained from searches that satisfy the following criteria will be excluded from the study.

- Projects estimation is mentioned (in the title or abstract), but it is not the main topic of the research;
- The study is not in the context of agile software development;
- The study is not focused on estimation during the early stages of the project life cycle;
- The study was already found in another digital library included in this study;
- The full text of the study is not available.

The exclusion criteria were applied mainly during the screening of the title, abstract, and full text of the results returned by the queries made with the search string in the selected digital libraries.

4.3.3. Quality Assessment

To evaluate the validity, reliability, and treatment of bias in the evidence presented by the selected studies, we applied a checklist to assess the quality of them. We used the quality evaluation criteria defined by Usman [19]; 10 evaluation questions were adapted as follows.

- QA1: Are the study objectives clearly specified?
- QA2: Is the study design consistent with the established objectives?
- QA3: Are the estimation approaches and/or techniques included in the study objectively described and compared?
- QA4: Are the methods and criteria used for data collection clearly described?
- QA5: Is the context in which estimation techniques are studied (agile methods, activities, type of software, experience) clearly defined?
- QA6: Is the purpose of data analysis clearly specified?
- QA7: Were statistical techniques used to analyze the data?
- QA8: Are any issues discussed about threats to the validity of the results?
- QA9: Is there an attempt to answer each research question established in the study?
- QA10: Were the findings clearly presented and supported by the results obtained?

The quality assessment was applied to the articles filtered by screening the title, abstract, and full text. During the quality assessment process, the first author performs a detailed analysis and assigns a response option for each question in the quality assessment criteria. The response options are “Yes”, “Partially”, or “No”. The answer “Yes” gives a score of 1, the answer “Partially” adds 0.2, while the answer “No” does not provide a score. It was established that articles must achieve a score equal to or greater than the first quartile of the values assigned to the studies during the quality assessment, similar to what was performed by Alsaadi et al. [12]. Secondary authors participate in the quality assessment through work sessions in which the responses assigned to several articles are randomly verified. The articles’ quality assessment results are in Table A1 in Appendix A.

4.4. Data Extraction

The data extraction activity involves thoroughly analyzing each of the selected primary studies. Data must be extracted from this analysis to allow for synthesis and reporting of the study results. A form was designed and implemented using a spreadsheet tool to extract the data. The fields of the form are aimed at capturing (1) general information about the study (such as year of publication, author, and type of study, among others) and (2) information that addresses the research questions of this study. A detailed description of the fields can be found in Tables A2 and A3 in Appendix A.

The first author initially performed the data extraction process. Subsequently, a series of work sessions were conducted. The secondary authors randomly chose some of the primary studies and performed data extraction again to verify them against the data obtained by the first author. Controversies that arose were resolved, and any identified errors were corrected.

4.5. Data Synthesis

We created a classification scheme to obtain a general and objective view of the information and facilitate data extraction, organization, and synthesis. This scheme was designed to create affinity diagrams proposed by Plain [41] and refined based on the classification scheme proposed by Usman [42].

The affinity diagram technique was initially used as a visual tool for organizing information to identify patterns, trends, or relationships between studies. The authors of this study participated in a series of work sessions during which they brainstormed, identified, and classified keywords into categories, generating a first approximation of the classification scheme.

Subsequently, the classification scheme was refined, taking Usman's proposal as a reference [42]. Usman used the taxonomy design method to organize and classify information related to estimation in ASD methods. The proposed scheme by Usman includes four main dimensions: Context, Techniques, Predictors, and Effort Estimate. Each dimension comprises facets with a set of possible values to be assigned.

The following section presents the results from the literature review study, which was conducted using the methodology described in this section.

5. Results

This study uses descriptive (narrative) synthesis to present the most outstanding findings from the review of the selected primary studies. These results are presented in the following subsections.

5.1. Search Results

We obtained 836 records by applying the search string and parameters in the selected digital libraries. We reviewed the titles and abstracts of these records to verify whether they met the above eligibility criteria. Then, we discarded 746 articles that did not fully or partially satisfy these criteria.

We obtained the full text of the remaining 90 articles and reviewed them to apply the eligibility criteria again, resulting in 22 selected studies for quality assessment.

In the next stage, after the quality assessment, four studies were excluded because they obtained a score lower than 5.8, corresponding to the first quartile of the values assigned to the studies included in the quality assessment. Thus, 18 primary studies were finally selected.

In Figure 2, the number of resulting studies is shown graphically through the different stages of the selection process.

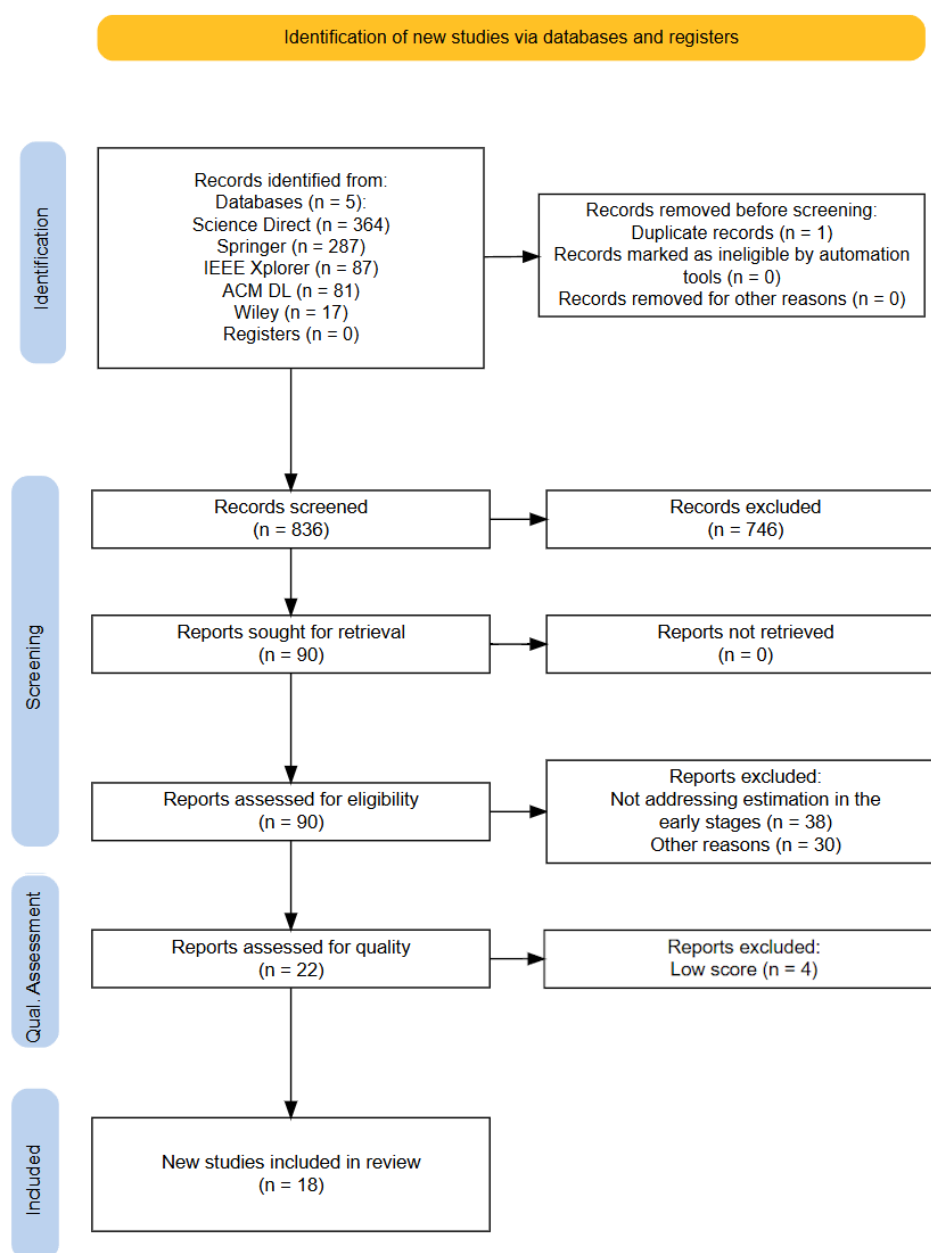


Figure 2. Results of the execution of the literature review.

It should be noted that articles were primarily excluded during the screening by title and abstract review phase because, although mentioned in the title or abstract, the study was not focused on project estimation.

In the screening by full-text review phase, the main reason for exclusion was that the studies did not address the issue of project estimation in the early stages of the life cycle, even though, in some cases, a reference was made to it in the article abstract. However, this mention was generally only to establish the importance or relationship of the project estimation with the topic addressed by that study.

Figure 3 graphically shows the number of primary studies selected from each digital library and their distribution across the years of publication. The number of studies selected per digital library is distributed almost equally between ScienceDirect and Springer, each contributing six primary studies, while IEEE Xplore contributed five. The Wiley Digital Library contributed a single primary study to the final selection, while the ACM Digital Library did not contribute any primary studies to the final selection.

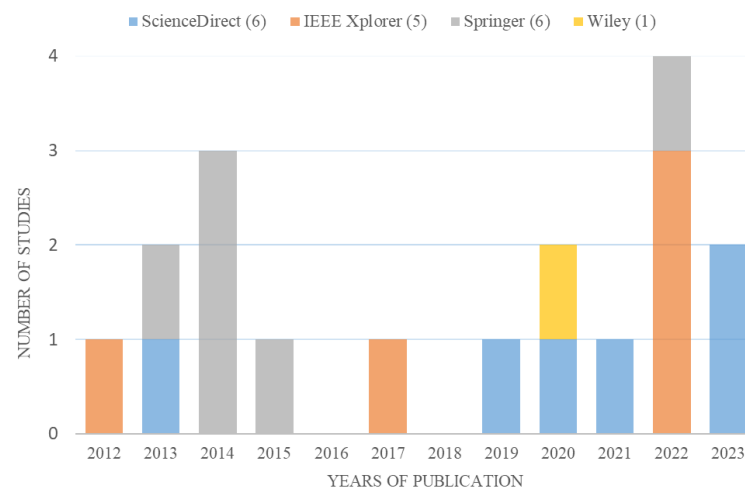


Figure 3. Distribution of primary studies by digital library and years of publication.

Sixty percent of the selected primary studies were published in a Journal, 33% in a Conference, and only one article (6%) in a Workshop.

Table 3 lists the selected primary studies showing the assigned ID, main author, year of publication, digital library from which it was obtained, and the publication type.

Table 3. Selected primary studies.

ID	Authors	Year	Library	Pub. Type	Ref.
PS01	Khatibi Bardsiri et al.	2014	Springer	Journal	[43]
PS02	Kaushik Anupama et al.	2022	Springer	Journal	[44]
PS03	Khatibi Bardsiri et al.	2013	Springer	Journal	[45]
PS04	Shaima Hameed et al.	2023	ScienceDirect	Journal	[46]
PS05	Przemysław Pospieszny et al.	2015	Springer	Conference	[47]
PS06	Ishrar Hussain et al.	2013	ScienceDirect	Journal	[20]
PS07	Philipp Hansen et al.	2022	IEEE Xplore	Conference	[48]
PS08	Wilson Rosa et al.	2023	ScienceDirect	Journal	[49]
PS09	Geng Liu et al.	2021	ScienceDirect	Journal	[50]
PS10	Wilson Rosa et al.	2017	IEEE Xplore	Conference	[25]
PS11	Fehlmann Thomas et al.	2014	Springer	Workshop	[51]
PS12	S. Malathi et al.	2014	Springer	Conference	[52]
PS13	Wilson Rosa et al.	2022	IEEE Xplore	Journal	[53]
PS14	Luigi Lavazza et al.	2019	ScienceDirect	Journal	[54]
PS15	Geng Liu et al.	2020	Wiley	Journal	[55]
PS16	Ziema Mushtaq et al.	2020	ScienceDirect	Journal	[56]
PS17	Ratnesh Litoriya et al.	2013	IEEE Xplore	Conference	[57]
PS18	Hüseyin Ünlü et al.	2022	IEEE Xplore	Conference	[58]

5.2. Classification Scheme

Based on the analysis of the selected primary studies, we found that the authors recommend using estimation techniques based on predictive models. In this context, a model is a mathematical formula or an algorithm that uses a set of input values, known as predictors [25,42], to calculate the project effort. Among the most commonly used methods to build these estimation models are Ordinary Least Squares Regression [49,53,55], Function Point Analysis [50,54], and Artificial Neural Networks [47].

The authors [20,25,43–47,49,50,52–57] employed databases containing metrics from previous projects to develop and evaluate their estimation models. These databases include quantitative and qualitative records of attributes and characteristics from past projects, such

as team experience, tools used (virtual machines, programming languages, development platforms), database size, software size, and application type.

The information in these databases is crucial for building estimation models. Therefore, the authors often specify certain characteristics regarding the origin of the data. For example, they indicate whether the data come from academic, industrial, or both types of projects [25,45–47,49,50,52–56,58]. They also detail whether the projects come from the same company (single-company) or include projects from different organizations (cross-company) [25,45–47,49,50,52,53,56,58].

Depending on their needs, the authors choose between public and private/proprietary databases. Public databases are available from organizational websites and can be used with few restrictions, while private or proprietary databases are for exclusive or limited use. Some authors [43–46,50,57] use multiple databases to ensure they have sufficient data.

Several studies [25,43,45–47,49,55,56] reported the need for preprocessing to normalize the data structure, especially when using different databases. Additionally, some values that are not directly recorded but can be derived from available data need to be calculated. For example, in some cases, effort expressed in hours is converted to effort expressed in Person-Months [25].

The authors [45–47] divide the records from the databases into two sets: one for constructing or training the estimation models and the other for evaluation. They [25,44,49,54] experiment with different metric combinations using the training dataset to achieve greater accuracy. Finally, they [25,43–47,49,50,53,55,56,58] assess the performance of the created models using metrics such as mean magnitude of relative error (MMRE) and percentage of prediction (PRED), comparing the results and statistically determining which model performs best, either among their own models or compared to those of other authors.

During model construction, the predictor values are obtained from database records. However, in practice, these values are specific to the project being estimated. Therefore, obtaining or calculating these values from the input artifacts available at the time of estimation is necessary [25,49,51,53,55,56,58].

Compared with Usman’s scheme [42], the classification scheme formulated for this study is focused on the context of the initial estimation of the entire project—that is, of all software development activities. The final result is presented through a graphical model in Figure 4.

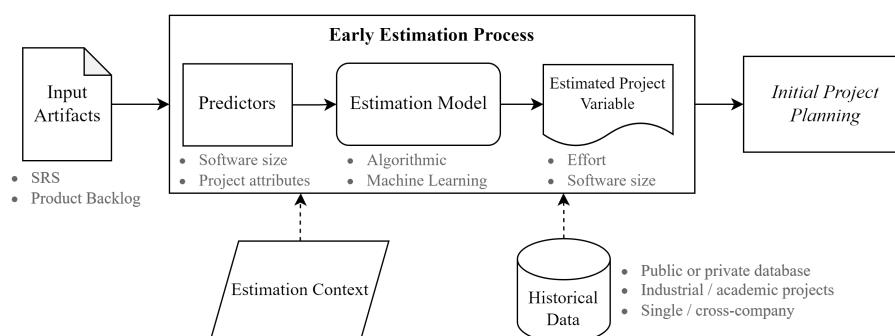


Figure 4. The classification scheme for estimating early stages of agile software development projects.

The classification scheme (Figure 4) represents the estimation process in its central box labeled “Early Estimation Process” to emphasize the estimation performed before starting the project for feasibility, budgeting, and planning purposes. Within this central box, from left to right, (1) the predictors (or independent variables) as inputs to (2) the estimation models with (3) the dependent variables as their output are shown. Compared to Usman’s model [42], which uses “Effort Predictors” as a dimension, this model uses the term “Predictors” in a more general way, as different estimation techniques can calculate the value of various dependent variables such as software size, cost, and duration, in addition to effort. Similarly, instead of using the “Effort Estimate” dimension, it generalizes with the “Estimated Project Variable” concept.

The input artifacts to the estimation process are represented on the left side, outside the central box. In Usman’s proposed model [42], the “Estimation Entity” facet is mentioned within the “Context” dimension. Usman defines “Estimation Entity” as “the inputs to the estimation process”, similar to the definition of input artifact in this study; although, in the model we propose, greater emphasis is given to it due to its importance during the initial estimates.

At the bottom, outside the central box, two elements that influence the estimation process are represented:

1. Estimation Context: represents specific characteristics under which the project is executed and largely coincides with the Context dimension of Usman’s model.
2. Historical Data: some estimation techniques use historical data to calibrate and evaluate their estimation models. Historical data are not represented in Usman’s model.

Finally, on the right side, outside the central box, the estimation result is shown to be mainly used for initial project planning.

5.3. Answer to Research Questions

This section answers the research questions posed for this study based on the results obtained from analyzing the data from the selected studies.

5.3.1. RQ1. What Input Artifacts Are Used to Estimate the Early Stages of Agile Software Development Projects?

Input artifacts refer to artifacts such as requirements, documents, diagrams, and models that are available during the project’s initial estimation. They contain relevant information about the features, functionalities, and constraints of the product to be built.

The results of this study showed that seven of the eighteen selected studies (PS06 [20], PS08 [49], PS10 [25], PS11 [51], PS13 [53], PS15 [55], and PS18 [58]) examined or specified input artifacts used to perform the project estimate. However, no further information was provided on the requirements’ specification level, which significantly influences the estimate’s accuracy. According to these studies, the primary input artifacts are the Software Requirements Specification (SRS), the Product Backlog, User Stories, and UML Models. Table 4 lists the input artifacts and the primary studies that examine them.

Table 4. List of input artifacts and primary studies that examine them.

Input Documents	Primary Studies
Textual requirements	PS06 [20]
Product backlog	PS08 [49], PS18 [58]
Software requirements specification	PS10 [25], PS13 [53]
User stories	PS11 [51]
UML models	PS15 [55]

Note that 11 studies (more than 61%) have not addressed this topic. Among these eleven studies, eight of them (PS01 [43], PS02 [44], PS03 [45], PS04 [46], PS05 [47], PS07 [48], PS12 [52], PS17 [57]) involve the Machine Learning approach.

According to the selected primary studies, the Software Requirements Specification (SRS) and the Product Backlog are the primary input artifacts for the estimation process in the early stages of the project. This is supported by the fact that, on the one hand, the textual requirements referred by Ishrar Hussain et al. in PS06 [20] are text fragments that can be found in the SRS or a Product Backlog Item (PBI); on the other hand, the User Stories that Thomas Fehlmann and Eberhard Kranich indicated in PS11 [51] are considered a PBI.

Depending on the process or methodology used for software development, UML models are generally developed in the project’s execution phase. Therefore, they are usually unavailable through an initial estimate for budgeting purposes, and generating them may involve additional time and effort.

5.3.2. RQ2. What Estimation Approaches and Techniques Are Used to Perform Estimation in the Early Stages of Agile Software Development Projects?

Estimation techniques refer to methods used to predict the value of variables of interest for budgeting and initial project planning. Different estimation approaches proposed over the years usually have characteristics that provide them with advantages but also generate some disadvantages, depending on the context in which they are applied [13].

According to the selected studies, data-driven techniques are predominant, with 89% of the primary studies employing them. Only a few studies (11%) use an expert-driven approach (PS07 [48] and PS11 [51]), and these also combine it with data-driven techniques, resulting in a hybrid approach. Half (PS08 [49], PS09 [50], PS10 [25], PS13 [53], PS14 [54], PS15 [55], PS16 [56], and PS18 [58]) apply algorithmic methods among the studies that utilize data-driven techniques. The remaining studies employ machine learning (PS02 [44], PS04 [46], PS05 [47], PS06 [20], and PS07 [48]) or combine machine learning with other algorithmic or non-algorithmic methods (PS01, PS03, PS12, and PS17), forming what is known as a hybrid technique.

Figure 5 shows the main estimation techniques used in the selected studies. According to the results, estimation techniques usually use regression models and function point analysis under the algorithmic approach. A greater diversity of strategies is used for machine learning, where data mining and fuzzy expert systems stand out. Finally, under the non-algorithmic approach, analogy-based estimation combined with machine learning or algorithmic methods is the only technique used in the selected studies.

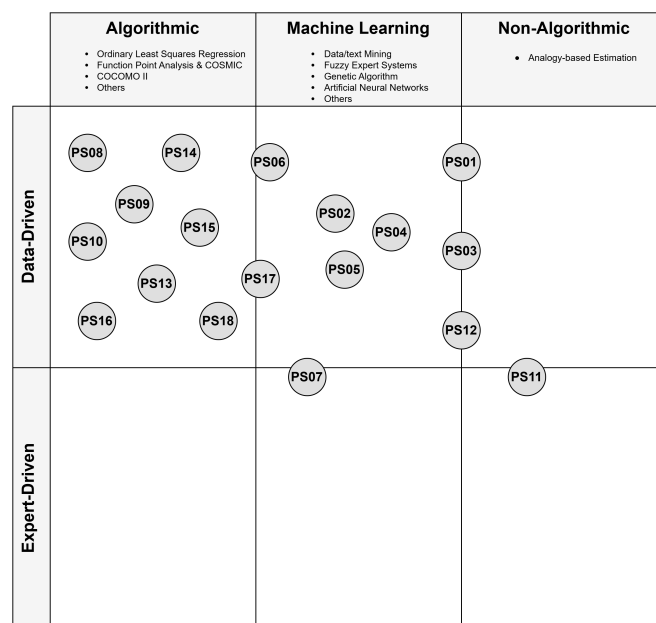


Figure 5. General classification of estimation techniques based on [30,59].

As part of research question RQ2, we expect to understand the authors' reasons or arguments for justifying the estimation technique they used. We abstracted and categorized the main arguments presented by the authors of the selected studies. These abstractions are listed below, along with the studies in which they can be identified.

- In PS04 [46], PS12 [52], PS14 [54], and PS17 [57], the authors refer to previous studies that demonstrated the effective use of the estimation technique they study.
- In PS05 [47] and PS07 [48], the authors dismiss expert-based approach techniques, stating that they are error-prone, often overly optimistic, and that experts entail a high cost for companies.

- The authors of PS01 [43], PS02 [44], PS03 [45], PS05 [47], and PS11 [51] indicate that algorithmic techniques are ineffective when handling data with no linear relationship; in this situation, machine learning methods are often effective.
- The authors in PS06 [20], PS08 [49], PS09 [50], PS10 [25], PS11 [51], PS13 [53], PS14 [54], PS15 [55], PS16 [56], and PS18 [58] indicate that software size is the primary predictor of project effort. They propose using a technique derived from some software size measurement method or new metrics for measuring software size that can be calculated from information available in the early stages of the project.

Based on the results obtained regarding the estimation approaches and techniques for projects' early stages, we can point out the following observations:

1. It primarily uses data-driven approaches to create linear regression and machine learning models;
2. Researchers propose or improve functional size measurement methods, such as COSMIC, so that they can be used to estimate the functional size of software in the early stages.

5.3.3. RQ3. What Predictors Are Used to Estimate in the Early Stages of Agile Development Projects?

Predictors refer to the independent variables that estimation techniques use to calculate the value of the dependent variables.

In this regard, we found that in 11 of the 18 selected primary studies, the software size was used as the main predictor. Five of these eleven studies use it combined with other predictors: in PS10 [25] and PS13 [53] with peak staff and super domain, in PS16 with mobile technical and environmental complexity factors, in PS11 with business factors (responsiveness, be compelling, friendliness, personalization, and competence), and in PS05 [47] with key project attributes (industry sector, application type, development type, development platform, language type, architecture, etc.). In fact, the key project attributes are the second most used predictor in the selected studies (PS01 [43], PS03 [45], and PS05 [47]). Additionally, in a pair of studies (PS02 [44] and PS17 [57]), the cost factors proposed in COCOMO II were used as independent variables.

The graph in Figure 6 compares the predictors and the approaches applied. This graph shows that software size is usually used as a predictor for algorithmic techniques. Machine learning is evenly distributed among the different predictors reported by the selected primary studies.

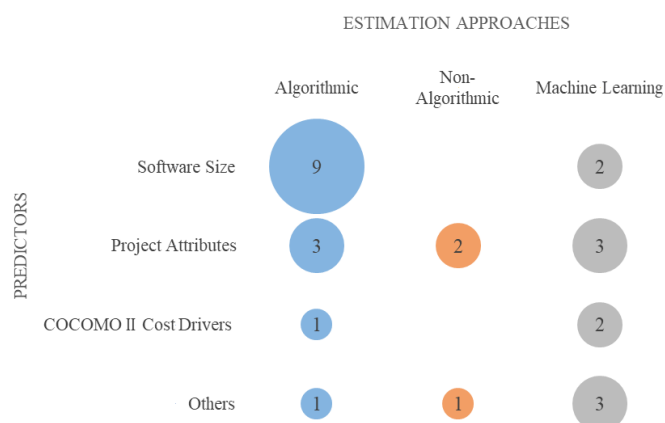


Figure 6. Comparison between estimation approaches and predictors used by the selected primary studies.

Nine studies (out of eleven) based on software size as a predictor use or compare Function Points (or some derivative, such as COSMIC Function Points) as a size metric. To a lesser extent, Story Points were used (in one study for comparison purposes) or a custom metric was proposed (in three studies: PS10 [25], PS13 [53], and PS18 [58]).

Some studies (PS06 [20], PS08 [49], PS10 [25], PS13 [53], PS17 [57], PS18 [58]) highlight the relevance of using predictors for which it is possible to obtain or calculate their value from information available in the early stages of the project. The authors propose software size as the primary predictor and use function points (or similar) as a metric. However, some authors have proposed new metrics for size, such as Initial Software Requirements (PS10 [25], PS13 [53]) and Event Points (PS18 [58]), arguing that these are easier to obtain in the early stages of the project.

5.3.4. RQ4. What Are the Characteristics of the Datasets Used by Estimation Techniques in the Early Stages of Agile Development Projects?

Algorithmic and machine learning techniques usually use data to create, calibrate, and validate the models they are based on.

Four of the data-driven estimation techniques (in 16 of the 18 selected studies) do not specify the origin of the used data. Seven studies use data from industrial projects, while five use a combination of industrial and academic project data.

Datasets can also be classified depending on whether they come from single- or cross-company projects. This classification is relevant because it impacts the homogeneity with which the data are measured and recorded. Among these datasets, only one study uses data from projects at a single company, while nine studies use data from multiple companies. The classification of the dataset they used was not specified in the six selected studies.

Another characteristic of the datasets used in the selected studies is whether the data were obtained from a public database or used data from their own (or private) projects. The graph in Figure 7 shows the distribution of these two types of databases for the estimation approaches.

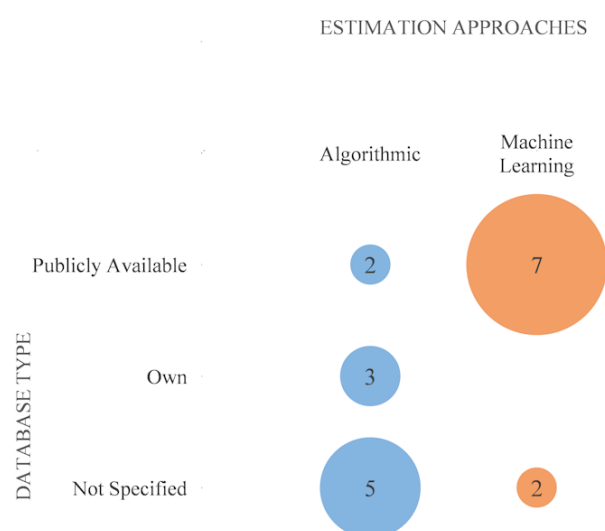


Figure 7. Approaches to estimation techniques versus the type of database used.

In algorithmic, whether the dataset was obtained from a publicly available database or if it is its own data has not been specified. On the other hand, publicly available databases are usually used for machine learning-based techniques.

6. Discussion

The focus of this research is on understanding the following facets of early-stage estimates in ASD projects:

- Input artifacts;
- Approaches (or classifications) of estimation techniques;
- Predictors (or independent variables);
- Datasets.

This section presents the main findings and the relationships among the selected primary studies regarding these facets. Trends are also analyzed and identified, and areas where further research is necessary are proposed.

Estimates are used during project execution to plan releases, iterations, or daily tasks; in this context, input artifacts are usually available and contain sufficient details about the software requirements. However, initial planning is characterized by a need for more information and clarity regarding the requirements for the product to be developed. Since requirements are one of the main inputs for estimation activities because they determine the size and complexity of the software to be built, it is imperative to understand what issues (related to input artifacts) have been addressed in research in this context.

One of the findings of this study is related to the use of proposed input artifacts for the estimation process. We found that in seven of the eighteen selected primary studies, input artifacts are used in size estimation methods (PS06 [20], PS08 [49], PS11 [51], and PS15 [55]) or as a software size metric (PS10 [25], PS13 [53], and PS18 [58]). Conversely, with regard to the eleven studies that do not mention or use input artifacts in their estimation techniques, eight of them (PS01 [43], PS02 [44], PS03 [45], PS04 [46], PS05 [47], PS07 [48], PS12 [52], PS17 [57]) are based on machine learning methods, primarily using project attributes (industry sector, application type, development type, development platform, language type, architecture, etc.) as predictors rather than software size.

Organizations typically implement or adapt different software development processes, methods, and practices, creating a hybrid development approach [60,61]; consequently, the artifacts they produce are different and developed at various moments in the project life cycle. This leads to two critical points identified in the results of this study:

1. The availability of input artifacts used during the early stages of the project life cycle is assumed. For example, User Stories (PS11 [51], PS18 [58]) and UML diagrams (PS15) are generally developed later during project execution. In contrast, other studies (PS06 [20], PS08 [49], and PS10 [25]) emphasize this condition, ensuring that their proposals are based on information available in the early stages of the project. This is important for future research addressing input artifacts as part of their study. From a practical perspective, it would be interesting to know which input artifacts are generally available for practitioners to use during early-stage estimates in ASD projects.
2. Other studies (PS06 [20], PS08 [49], and PS10 [25]) highlight the importance of homogeneity, style, and quality with which input artifacts are documented to make their proposals effective. Ishrar Hussain et al., in PS06 [20], suggest that each organization must calibrate the estimation method they propose to the characteristics of their input artifacts. For this reason, researchers and practitioners must consider that estimation methods must be adjusted to each organization's particularities to achieve positive results.

Further, some studies suggest techniques that require processing of the artifacts to use them in the estimation; for example, in PS06, requirements must be classified by functional processes, while in PS15 [55], UML diagrams must be elaborated according to the ICONIX process. Authors need to consider the additional effort these activities entail. For this reason, it is crucial to have an overview of the average resources (people and time) that organizations invest in project estimation when these have not yet been awarded or approved and how much additional effort they are willing to invest in the estimation process if, in return, they can obtain more accurate estimates.

Various literature reviews on estimation in agile projects [19,33,34] reveal that expert-driven approaches, such as Expert Judgment, Planning Poker, and Use Case Points, among others, are the most investigated by authors for planning at the release, itera-

tion, or daily level. Conversely, we found that data-driven approaches are the most used for early-stage estimates, mainly because of the following:

- These allow for avoiding the bias generated by expert-driven approaches;
- As the authors argue, it is convenient to use the results of previous projects to predict new ones.

The need for historical project databases characterizes data-driven approaches. This leads to two strategies for designing, calibrating, and evaluating estimation techniques: (a) the use of publicly available databases and (b) the use of proprietary databases. In this regard, we identified the following trends in primary studies:

1. Studies using machine learning-based techniques commonly use publicly available databases. This allows them to access a large amount of data, which is essential for designing models and facilitates evaluation and comparison against similar proposals. However, these databases contain records of projects executed in the 1980s and 1990s. Therefore, they are considered obsolete, as software development's context (such as global and remote development), technologies (mobile and cloud platforms), and practices (Agile, DevOps) have evolved significantly since then [43,47]. The effort involved in developing a particular software product using technologies and methodologies from decades ago differs from the effort involved under current conditions. While organization-specific databases can offer significant benefits, if authors choose publicly available databases to obtain more records, future studies should consider using or creating publicly available databases with more recent agile project records that contain sufficient project context information and use standardized metrics [19].
2. Studies using algorithmic techniques typically use proprietary historical databases. Using data from projects with the same or similar context and with the same metrics allows for generating regression models with better correlation or software size measurements of projects with similar characteristics (considering that linear regression and software size measurement methods are the most used algorithmic techniques, as found in the results; see Figure 5). However, some organizations may not have available records of previous projects. In this situation, future research could focus on integrating tools into the estimation process that allow (semi-) automating metrics collection from project artifacts (requirements, source code, and defect reports, among others) so that organizations can easily and quickly build their historical databases. For example, measuring the size of the software using a tool that counts the Function Points or Use Case Points based on the automatic analysis of key elements available in the source code.

The results of this study have revealed that the vast majority of primary studies have opted for algorithmic and machine-learning techniques. Only a few studies (PS01 [43], PS03 [45], PS12 [52], and PS17 [57]) have used a hybrid technique. Some authors [59,62] have suggested that there is no universal technique for estimating software projects. Instead, combining estimation techniques under different approaches could allow more accurate results. Hence, a topic for future research could be identifying the primary deficiencies or problems that algorithmic and machine-learning techniques (generally data-driven) usually have in early-stage estimates and determining whether expert-driven approaches can address these deficiencies.

Similarly, literature review studies on estimation in agile projects [19,34] have found that Story Points and Use Case Points are the most commonly used metrics for software size when planning at the release, iteration, or daily level. Generally, at these planning levels, there is a better perspective on the activities to be carried out to address the requirements. Story Points estimation is preferred at these project stages, later converted into hours or days of work. However, Story Points are a relative and subjective measurement, so historical records using this metric are not helpful when used on a different project. Conversely, in the early stages of the project, it is challenging to determine activities at such a detailed level; thus, some proposed estimation techniques opt to estimate software size, considering

that project effort is closely linked to software size [20,49]. This situation is reflected in the results of this study, where 11 of the 18 selected primary studies use software size as the main predictor, mainly proposing algorithmic techniques that use or adapt software functional size measurement methods such as COSMIC Function Points and create models based on statistical techniques like Ordinary Least Squares Regression.

Despite the extensive literature available on software project estimation, there are still important topics to cover regarding early-stage estimation in ASD projects. Work related to agile projects has mainly focused on planning at the release, iteration, and daily levels. Still, as presented in this section, early-stage research presents different conditions that lead to the use of other solutions.

7. Threats to Validity

This section analyzes the validity of the results obtained and the findings presented in this study. Although this study was based on a widely used methodology in software engineering that helps reduce bias, certain aspects of its execution may threaten its validity. Below, we present some of the threats identified and how we mitigated them.

- The search string was initially constructed from the main terms of the research questions. However, a limited number of results led to refining the search string. Synonyms and related terms were identified based on test searches, allowing us to construct a search string that yielded more results.
- As part of the inclusion and exclusion criteria for study selection, special emphasis was placed on early-stage agile project estimation studies. In some cases, determining whether a study addressed early-stage estimation was challenging. To avoid incorrectly discarding relevant studies, those in which there was uncertainty were flagged for consensus review by all authors, who then judged their inclusion.
- The quality assessment of the studies was based on the questions from Usman et al. [19]. The scores assigned to each study involved a degree of subjectivity. To reduce bias, all authors independently participated in the assessment.
- The digital libraries used in the search process were not randomly chosen. They were selected based on an analysis of previous similar studies, identifying those libraries that yielded a significant number of relevant results.
- The selected studies do not use the same estimation terms and concepts, so it is common for authors to report their results in a non-homogeneous manner [3]. Techniques and references to previous studies were applied to generate a classification scheme for extracting, classifying, and analyzing information to present the authors' wide variety of reports.
- The authors of this study have approximately 20 years of experience in the software industry (including estimating and developing projects using the agile approach). This experience and their research background in software engineering helped reduce bias in the results, findings, and proposals presented.

8. Conclusions

In this article, we present a systematic mapping review about early estimation in agile software development projects. The results show that in the literature, it is preferred to use data-driven estimation techniques over expert-based techniques, arguing that the latter are prone to participant bias and tend to produce results with variable accuracy. However, expert-based techniques can help reduce the uncertainty caused by the limited information available about the product at the beginning of the project. Researchers and practitioners should consider a hybrid approach, combining techniques with different focuses to leverage the advantages of each while compensating for their shortcomings. This approach could lead to more accurate results.

Most of the estimation techniques found are based on predictive models built from data from previous projects, primarily using statistical methods such as linear regression and machine learning algorithms. A drawback identified in this process is that some

articles use data from projects carried out nearly four decades ago. Since technologies, tools, and software development methodologies have evolved significantly since then, the independent variables influencing project effort may differ significantly from current project conditions.

Another relevant aspect identified in the literature is the consideration of the context in which the estimation occurs at the beginning of the project. This context is characterized by limited information and ambiguity regarding the software requirements. Therefore, estimation models should avoid relying on unavailable information and artifacts during this phase. For example, if an estimation technique requires UML diagrams, these are generally developed during project execution, making it unsuitable to include them in an initial project estimation technique. The results also suggest that estimation processes should be dynamic, adapting to changes and project circumstances, and receiving feedback from the results obtained.

9. Future Directions

Future research should consider using more recent project datasets to develop predictive models that align with contemporary project contexts. Building predictive models requires a large amount of historical data, which is often not available because agile methods are characterized by generating minimal documentation and prioritizing programming-related activities, as suggested by the values and principles of the Agile Manifesto. One option to address this situation could be by automating these tasks through software tools. For example, the functional size of the software could be obtained through software tools to analyze the source code of previous projects.

When developing predictive models for early estimation, it is crucial to use data derived from the artifacts available at the beginning of the project and the information they contained at that time. Since some artifacts evolve and their information becomes more detailed and accurate as the project progresses, relying on later-stage information to generate predictive models could result in models with limited practical applicability.

It is essential for future research to investigate how expert-driven approaches can help when historical data are lacking, there is a high degree of uncertainty, or other aspects are present that data-driven approaches cannot predict (hybrid approach). Additionally, articles on software project estimation should report details about the context under which the results of their empirical studies were obtained so that the findings can be effectively applied in practice or serve as a basis for future research. Building updated historical project databases using standardized metrics and sufficient context information is also crucial for advancing the field.

Author Contributions: Conceptualization, methodology, formal analysis, investigation, writing original draft, writing—review and editing, visualization, J.G.R.I.; conceptualization, methodology, writing—original draft, writing—review and editing, supervision, project administration, G.B.; validation, data curation, writing—original draft, writing—review and editing, R.R.P. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the “Programa de Fortalecimiento a la Investigación 2024” (project numbers: PROFAPI CA_2024, PROFAPI 2024_099 and PROFAPI 2024_064) of the Instituto Tecnológico de Sonora.

Data Availability Statement: Not applicable.

Acknowledgments: This work was supported by the National Council of Humanity, Science and Technology (whose acronym in Spanish is CONAHCYT) of Mexico, with a scholarship assigned to the first author with identification no. CVU 206549.

Conflicts of Interest: The authors declare no conflict of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript, or in the decision to publish the results.

Appendix A

Table A1. Results of the Quality Assessment Checklist adopted from Kitchenham.

ID	QA1	QA2	QA3	QA4	QA5	QA6	QA7	QA8	QA9	QA10	Total
S1	Y	Y	P	Y	Y	Y	Y	Y	Y	Y	9.2
S2	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	10
S3	P	Y	Y	Y	Y	Y	Y	P	P	Y	7.6
S4	Y	Y	P	Y	P	P	P	N	Y	Y	5.8
S5	Y	Y	P	Y	P	Y	P	N	P	Y	5.8
S6	Y	Y	P	Y	Y	Y	Y	Y	Y	Y	9.2
S7	Y	Y	P	Y	P	Y	Y	N	P	Y	6.6
S8	Y	Y	P	Y	Y	P	P	N	P	Y	5.8
S9	Y	Y	P	Y	P	Y	Y	Y	P	Y	7.6
S10	Y	Y	Y	P	Y	P	Y	P	P	Y	6.8
S11	Y	Y	P	Y	P	Y	Y	Y	Y	Y	8.4
S12	Y	P	P	Y	P	Y	Y	N	Y	Y	6.6
S13	Y	Y	Y	P	P	Y	P	P	Y	Y	6.8
S14	Y	Y	Y	P	P	Y	Y	N	P	Y	6.6
S15	Y	Y	Y	Y	P	Y	Y	Y	Y	Y	9.2
S16	Y	Y	P	Y	P	Y	Y	N	Y	Y	7.4
S17	Y	Y	P	Y	P	Y	Y	Y	Y	Y	8.4
S18	Y	Y	Y	P	Y	Y	Y	Y	Y	Y	9.2

Table A2. General information form fields used during article selection.

Field	Description	Values
ID	Unique identifier of the article.	S1, S2, ..., Sn
Source	The name of the digital library from which the article was obtained.	ACM Digital Library, Science Direct (Elsevier), IEEE Xplorer, Springer, or Wiley
Title	Article title.	Text
Primary author	The name of the primary author of the article.	Text
Secondary authors	List of the names of the article's secondary authors.	Text list
Year	The year the article was published.	Numeric
Country	Country of the primary author of the article.	Text
Study type	The type of study conducted.	Empirical Study, Theoretical Study, SLR, etc.
Publication type	The type of publication in which the article was disseminated.	Journal, Conference, Workshop
Keywords	List of keywords listed in the article.	Text list
Objective of the study	Fragment of the article text in which the author describes the study's main objective.	Text
Research questions	A field for each research question was included to establish a subjective level at which the research question can be answered.	Yes, No, or Partially.

Table A3. Research questions form fields used during article selection.

Question	Field	Description	Values
RQ1	Documents	List of document names used during the initial estimate.	Name of each document.
RQ2	Approach	List of estimation approaches used during initial estimation.	Algorithmic, Non-algorithmic, Machine learning.
RQ2	Estimation technique	List the names of specific estimation techniques used, analyzed, or compared.	Text list (Linear Regression, Data Mining, Genetic Algorithm, Expert Judgment, etc.).
RQ2	Based on	If the estimation techniques used, analyzed, or compared are data-driven or expert-driven.	Data-Driven or Expert-Driven.

Table A3. Cont.

Question	Field	Description	Values
RQ2.1	Dependent variables	Names of the dependent variables being estimated by the techniques employed, analyzed, or compared.	Text list (Effort, Duration, Time, etc.).
RQ3	Independent variables	Names of the independent variables used by the estimation techniques used, analyzed, or compared.	Text list (Software Size, Project Attributes, etc.).
RQ3	Independent Variable Categories	List of independent variables categories.	People, Product, Technical, or Project
RQ3.1	Size metrics	Names of size metrics used to represent software size.	Text list (Function Points, Story Points, Use Case Points, etc.).
RQ4	Number of case studies	The number of case studies used in the creation and/or validation of the estimation model.	Numeric
RQ4	Origin	Indicates the type of project from which data were obtained.	Industrial, Academic, Mixed, or Not Specified.
RQ4	Type	Indicates if the data come from a single-company or cross-company.	Single-company, Cross-company, or Not Specified.
RQ4	Domain	Indicates the business domain from which the data were acquired.	Text list (Financial, Government, Military, etc.).
RQ4	Databases	List of the names of the databases from which data were obtained.	Text list (ISGB, COCOMO81, MAXWELL, NASA93, etc.).

References

- Gartner. *Gartner Forecasts Worldwide IT Spending to Grow 4.3% in 2023*; Gartner: Stamford, CN, USA, 2023.
- Ibraigheeth, M.; Fadzli, S. Core Factors for Software Projects Success. *JOIV Int. J. Inform. Vis.* **2019**, *3*, 69–74. [\[CrossRef\]](#)
- Grimstad, S.; Jørgensen, M.; Moløkken-Østfold, K. Software effort estimation terminology: The tower of Babel. *Inf. Softw. Technol.* **2006**, *48*, 302–310. [\[CrossRef\]](#)
- Cerpa, N.; Verner, J.M. Why did your project fail? *Commun. ACM* **2009**, *52*, 130–134. [\[CrossRef\]](#)
- Nasir, M.H.N.; Sahibuddin, S. Critical success factors for software projects: A comparative study. *Sci. Res. Essays* **2011**, *6*, 2174–2186.
- Sudhakar, G.P. A model of critical success factors for software projects. *J. Enterp. Inf. Manag.* **2012**, *25*, 537–558. [\[CrossRef\]](#)
- Aldahmash, A.; Gravell, A.; Howard, Y. A Review on the Critical Success Factors of Agile Software Development. In *Proceedings of the Systems, Software and Services Process Improvement*, Ostrava, Czech Republic, 6–8 September 2017; Volume 8, pp. 504–512. [\[CrossRef\]](#)
- Mahmood, Y.; Kama, N.; Azmi, A. A systematic review of studies on use case points and expert-based estimation of software development effort. *J. Softw. Evol. Process* **2020**, *32*, e2245. [\[CrossRef\]](#)
- Bogopa, M.E.; Marnewick, C. Critical success factors in software development projects. *S. Afr. Comput. J.* **2022**, *34*, 1–34. [\[CrossRef\]](#)
- Kotowaroo, M.; Sungkur, R., Success and Failure Factors Affecting Software Development Projects from IT Professionals' Perspective. In *Soft Computing for Security Applications*; Springer Nature: Singapore, 2022; pp. 757–772. [\[CrossRef\]](#)
- PMI. (Ed.) *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 5th ed.; Project Management Institute: Newtown Township, PA, USA, 2013.
- Alsaadi, B.; Saeedi, K. Data-driven effort estimation techniques of agile user stories: A systematic literature review. *Artif. Intell. Rev.* **2022**, *55*, 5485–5516. [\[CrossRef\]](#)
- McConnell, S. *Software Estimation: Demystifying the Black Art*; Microsoft Press: Redmond, WA, USA, 2006.
- Suri, P.; Ranjan, P. Comparative Analysis of Software Effort Estimation Techniques. *Int. J. Comput. Appl.* **2012**, *48*, 975–8887.
- Peters, K. Software Project Estimation. *Methods Tools Glob. Knowl. Source Softw. Dev. Prof.* **2000**, *8*, 2–15.
- Prakash, B.; Viswanathan, V. A Survey on Software Estimation Techniques in Traditional and Agile Development Models. *Indones. J. Electr. Eng. Comput. Sci.* **2017**, *7*, 867–876. [\[CrossRef\]](#)
- Beck, K.; Beedle, M.; van Bennekum, A.; Cockburn, A.; Cunningham, W.; Fowler, M.; Grenning, J.; Highsmith, J.; Hunt, A.; Jeffries, R.; et al. Manifesto for Agile Software Development. Agile Alliance. 2001. Available online: <https://agilemanifesto.org/> (accessed on 29 October 2024).
- Mallidi, R.K.; Sharma, M. Study on Agile Story Point Estimation Techniques and Challenges. *Int. J. Comput. Appl.* **2021**, *174*, 9–14. [\[CrossRef\]](#)

19. Usman, M.; Mendes, E.; Weidt, F.; Britto, R. Effort Estimation in Agile Software Development: A Systematic Literature Review. In Proceedings of the Proceedings of the 10th International Conference on Predictive Models in Software Engineering. Association for Computing Machinery, Turin, Italy, 17 September 2014; pp. 82–91. [\[CrossRef\]](#)
20. Hussain, I.; Kosseim, L.; Ormandjieva, O. Approximation of COSMIC functional size to support early effort estimation in Agile. *Data Knowl. Eng.* **2013**, *85*, 2–14. [\[CrossRef\]](#)
21. Bisikirskienė, L.; Čeponienė, L.; Jurgelaitis, M.; Ablonskis, L.; Grigonytė, E. Compiling Requirements from Models for Early Phase Scope Estimation in Agile Software Development Projects. *Appl. Sci.* **2023**, *13*, 2353. [\[CrossRef\]](#)
22. Coelho, E.; Basu, A. Effort Estimation in Agile Software Development using Story Points. *Int. J. Appl. Inf. Syst.* **2012**, *3*, 7–10. [\[CrossRef\]](#)
23. Bloch, M.; Blumberg, S.; Laartz, J. Delivering large-scale IT projects on time, on budget, and on value. *Harv. Bus. Rev.* **2012**, *5*, 2–7.
24. Nassif, A.B.; Ho, D.; Capretz, L.F. Towards an early software estimation using log-linear regression and a multilayer perceptron model. *J. Syst. Softw.* **2013**, *86*, 144–160. [\[CrossRef\]](#)
25. Rosa, W.; Madachy, R.; Clark, B.; Boehm, B. Early Phase Cost Models for Agile Software Processes in the US DoD. In Proceedings of the 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), Toronto, ON, Canada, 9–10 November 2017; pp. 30–37. [\[CrossRef\]](#)
26. Hoy, Z.; Xu, M. Agile Software Requirements Engineering Challenges-Solutions—A Conceptual Framework from Systematic Literature Review. *Information* **2023**, *14*, 322. [\[CrossRef\]](#)
27. Cohn, M. *Agile Estimating and Planning*; Prentice Hall: Hoboken, NJ, USA, 2005.
28. Jørgensen, M. Forecasting of software development work effort: Evidence on expert judgement and formal models. *Int. J. Forecast.* **2007**, *23*, 449–462. [\[CrossRef\]](#)
29. Moløkken, K.; Jørgensen, M. A review of software surveys on software effort estimation. In Proceedings of the 2003 International Symposium on Empirical Software Engineering, ISESE 2003, Rome, Italy, 30 September–1 October 2003; pp. 223–230.
30. Vera, T.; Ochoa, S.; Perovich, D. *Survey of Software Development Effort Estimation Taxonomies*; Computer Science Department, University of Chile: Santiago, Chile, 2018. [\[CrossRef\]](#)
31. Britto, R.; Usman, M.; Mendes, E. Effort Estimation in Agile Global Software Development Context. In Proceedings of the Agile Methods. Large-Scale Development, Refactoring, Testing, and Estimation, Rome, Italy, 26–30 May 2014; Dingsoyr, T., Moe, N.B., Tonelli, R., Counsell, S., Gencel, C., Petersen, K., Eds.; Springer International Publishing: Berlin/Heidelberg, Germany, 2014; pp. 182–192.
32. Bilgaiyan, S.; Sagnika, S.; Mishra, S.; Das, M.N. A Systematic Review on Software Cost Estimation in Agile Software Development. *J. Eng. Sci. Technol. Rev.* **2017**, *10*, 51–64. [\[CrossRef\]](#)
33. Dantas, E.; Perkusich, M.; Dilenzo, E.; Santos, D.F.S.; Almeida, H.; Perkusich, A. Effort Estimation in Agile Software Development: An Updated Review. *Int. J. Softw. Eng. Knowl. Eng.* **2018**, *28*, 1811–1831. [\[CrossRef\]](#)
34. Fernández-Diego, M.; Méndez, E.R.; González-Ladrón-De-Guevara, F.; Abrahão, S.; Insfran, E. An Update on Effort Estimation in Agile Software Development: A Systematic Literature Review. *IEEE Access* **2020**, *8*, 166768–166800. [\[CrossRef\]](#)
35. Carbonera, C.E.; Farias, K.; Bischoff, V. Software development effort estimation: A systematic mapping study. *IET Softw.* **2020**, *14*, 328–344. [\[CrossRef\]](#)
36. Tandon, P.; Suman, U.; Rathore, M. A Systematic Literature Review on Effort Estimation in Agile Software Development using Machine Learning Techniques. *Int. J. Comput. Appl.* **2022**, *184*, 15–23. [\[CrossRef\]](#)
37. Peluffo-Ordóñez, D.; Martínez, L.S.; Timana, J.; Piñeros, C. Effort Estimation in Agile Software Development: A Systematic Map Study. *Inge CUC* **2023**, *14*, 22–36. [\[CrossRef\]](#)
38. Azzeh, M.; Nassif, A.B.; Attili, I.B. Predicting software effort from use case points: A systematic review. *Sci. Comput. Program.* **2021**, *204*, 102596. [\[CrossRef\]](#)
39. Kitchenham, B.; Charters, S. *Guidelines for performing Systematic Literature Reviews in Software Engineering*; EBSE Technical Report EBSE-2007-01; EBSE: Rio de Janeiro, Brazil, 2007.
40. Petersen, K.; Vakkalanka, S.; Kuzniarz, L. Guidelines for conducting systematic mapping studies in software engineering: An update. *Inf. Softw. Technol.* **2015**, *64*, 1–18. [\[CrossRef\]](#)
41. Plain, C. Build an affinity for KJ method. *Qual. Prog.* **2007**, *40*, 88.
42. Usman, M.; Börstler, J.; Petersen, K. An Effort Estimation Taxonomy for Agile Software Development. *Int. J. Softw. Eng. Knowl. Eng.* **2017**, *27*, 641–674. [\[CrossRef\]](#)
43. Bardsiri, V.K.; Jawawi, D.N.A.; Hashim, S.Z.M.; Khatibi, E. A flexible method to estimate the software development effort based on the classification of projects and localization of comparisons. *Empir. Softw. Eng.* **2014**, *19*, 857–884. [\[CrossRef\]](#)
44. Kaushik, A.; Singal, N. A hybrid model of wavelet neural network and metaheuristic algorithm for software development effort estimation. *Int. J. Inf. Technol.* **2022**, *14*, 1689–1698. [\[CrossRef\]](#)
45. Bardsiri, V.K.; Jawawi, D.N.A.; Hashim, S.Z.M.; Khatibi, E. A PSO-based model to increase the accuracy of software development effort estimation. *Softw. Qual. J.* **2013**, *21*, 501–526. [\[CrossRef\]](#)
46. Hameed, S.; Elsheikh, Y.; Azzeh, M. An optimized case-based software project effort estimation using genetic algorithm. *Inf. Softw. Technol.* **2023**, *153*, 107088. [\[CrossRef\]](#)

47. Pospieszny, P.; Czarnacka-Chrobot, B.; Kobylński, A. Application of Function Points and Data Mining Techniques for Software Estimation—A Combined Approach. In Proceedings of the Software Measurement, Kraków, Poland, 5–7 October 2015; Kobylński, A., Czarnacka-Chrobot, B., Świerczek, J., Eds.; Springer International Publishing: Berlin/Heidelberg, Germany, 2015; pp. 96–113.
48. Hansen, P.; Timinger, H. Concept of a Fuzzy Expert System for Story Point Estimations in Agile Projects. In Proceedings of the 2022 IEEE 28th International Conference on Engineering, Technology and Innovation (ICE/ITMC), Nancy, France, 19–23 June 2022; pp. 1–9. [\[CrossRef\]](#)
49. Rosa, W.; Jardine, S. Data-driven agile software cost estimation models for DHS and DoD. *J. Syst. Softw.* **2023**, *203*, 111739. [\[CrossRef\]](#)
50. Liu, G.; Lavazza, L. Early and quick function points analysis: Evaluations and proposals. *J. Syst. Softw.* **2021**, *174*, 110888. [\[CrossRef\]](#)
51. Michael, E.F.T.; Kranich. Early Software Project Estimation the Six Sigma Way. In Proceedings of the Agile Methods. Large-Scale Development, Refactoring, Testing, and Estimation, Rome, Italy, 26–30 May 2014; Dingsøyr, T., Moe, N.B., Tonelli, R., Counsell, S., Gencel, C., Petersen, K., Eds.; Springer International Publishing: Berlin/Heidelberg, Germany, 2014; pp. 193–208.
52. Malathi, S.; Sridhar, S. Effort Estimation in Software Cost Using Team Characteristics Based on Fuzzy Analogy Method—A Diverse Approach. In Proceedings of the Signal Processing and Information Technology, Dubai, UAE, 20–21 September 2012; Das, V.V., Elkafrawy, P., Eds.; Springer International Publishing: Berlin/Heidelberg, Germany, 2014; pp. 1–8.
53. Rosa, W.; Clark, B.K.; Madachy, R.; Boehm, B.W. Empirical Effort and Schedule Estimation Models for Agile Processes in the US DoD. *IEEE Trans. Softw. Eng.* **2022**, *48*, 3117–3130. [\[CrossRef\]](#)
54. Lavazza, L.; Morasca, S. Empirical evaluation and proposals for bands-based COSMIC early estimation methods. *Inf. Softw. Technol.* **2019**, *109*, 108–125. [\[CrossRef\]](#)
55. Liu, G.; Lavazza, L.; Tosi, D. Evolution of functional size measures through ICONIX process phases. *J. Softw. Evol. Process* **2020**, *32*, e2240. [\[CrossRef\]](#)
56. Mushtaq, Z.; Wahid, A. Inclusion of Functional and Non-Functional Parameters for the Prediction of Overall Efforts of Mobile Applications. *Comput. Stand. Interfaces* **2020**, *71*, 103404. [\[CrossRef\]](#)
57. Litoriya, R.; Sharma, N.; Kothari, A. Incorporating Cost driver substitution to improve the effort using Agile COCOMO II. In Proceedings of the 2012 CSI Sixth International Conference on Software Engineering (CONSEG), Indore, India, 5–7 September 2012; pp. 1–7. [\[CrossRef\]](#)
58. Ünlü, H.; Hacaloglu, T.; Büber, F.; Berrak, K.; Leblebici, O.; Demirörs, O. Utilization of Three Software Size Measures for Effort Estimation in Agile World: A Case Study. In Proceedings of the 2022 48th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), Gran Canaria, Spain, 31 August–2 September 2022; pp. 239–246. [\[CrossRef\]](#)
59. Sudarmaningtyas, P.; Mohamed, R.B. A Review Article on Software Effort Estimation in Agile Methodology. *Pertanika J. Sci. Technol.* **2021**, *29*, 837–861. [\[CrossRef\]](#)
60. Kuhrmann, M.; Diebold, P.; Münch, J.; Tell, P.; Garousi, V.; Felderer, M.; Trektre, K.; McCaffery, F.; Linssen, O.; Hanser, E.; et al. Hybrid Software and system development in practice: Waterfall, Scrum, and beyond. In Proceedings of the ICSSP '17: Proceedings of the 2017 International Conference on Software and System Process, Paris, France, 5–7 July 2017; Volume 6. [\[CrossRef\]](#)
61. Tell, P.; Klünder, J.; Küpper, S.; Raffo, D.; MacDonell, S.G.; Münch, J.; Pfahl, D.; Linssen, O.; Kuhrmann, M. What are Hybrid Development Methods Made Of? An Evidence-Based Characterization. In Proceedings of the 2019 IEEE/ACM International Conference on Software and System Processes (ICSSP), Montreal, QC, Canada, 25 May 2019; pp. 105–114. [\[CrossRef\]](#)
62. Zarour, A.; Zein, S. Software development estimation techniques in industrial contexts: An exploratory multiple case-study. *Int. J. Technol. Educ. Sci.* **2019**, *3*, 72–84.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.